Sport Manager

Débrief de fin de journée — 10 novembre 2026

Architecture web + mobile · preuves de fonctionnement · V1 de l'application web MVC

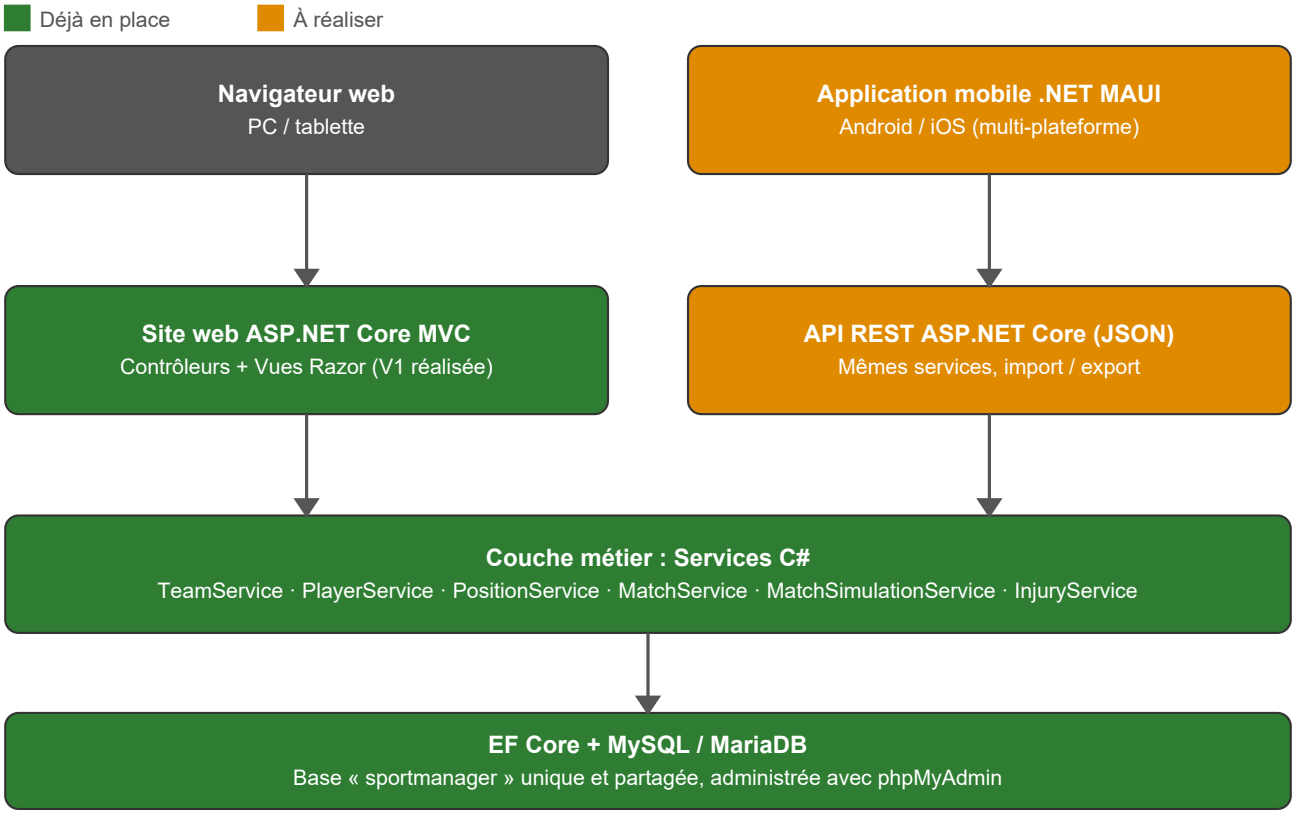
Livrable demandé	État
Choix d'architecture (web + mobile)	Défini : web ASP.NET Core MVC, mobile .NET MAUI via une API REST, services et base communs. **Le choix mobile est à valider.**
Preuves de fonctionnement de l'application actuelle (bloc 2)	Captures de toutes les pages, y compris une simulation de match avec blessure aléatoire.
V1 de l'application web MVC (sans authentification)	Réalisée : liste des équipes, liste des joueurs, détail d'une équipe, détail d'un joueur.

Contexte

Le projet Sport Manager (jeu de gestion d'équipes de sport avec simulation de matchs et blessures) existait sous la forme d'une application **Blazor Server**. Le cahier des charges demande une application **web MVC** (Modèle / Vue / Contrôleur), puis une application **mobile multi-plateforme** utilisant une API, avec une gestion multijoueur et l'import / export des données. Aujourd'hui, l'existant a été conservé et une première version MVC a été ajoutée à côté.

1. Choix d'architecture (web + mobile)

Schéma cible. Les blocs verts existent déjà, les blocs orange restent à réaliser.



Décisions

Sujet	Choix	Justification
Web	ASP.NET Core **MVC** (.NET 10)	Imposé par le cahier des charges. Même projet que l'existant : les services, les modèles et la base sont réutilisés.
Mobile	**.NET MAUI** (proposition, à valider)	Un seul code C# pour Android et iOS, même langage que le serveur, modèles de données partageables. Le mobile n'accède jamais à la base directement.
Communication	**API REST** (JSON) ASP.NET Core	Le mobile et le web utilisent les mêmes services métier : mêmes règles, mêmes données, et c'est ce qui rend le multijoueur possible.
Données	**MySQL / MariaDB** (base « sportmanager », administrée avec **phpMyAdmin**) + Entity Framework Core	Un vrai serveur de base de données, au lieu d'un fichier : une base unique côté serveur, partagée par tous les joueurs et par le web comme par l'API, avec des accès simultanés gérés par le serveur. Le passage depuis SQLite n'a demandé que le changement du fournisseur EF Core (**MySql.EntityFrameworkCore** 10.0.12) et de la chaîne de connexion. Les tables et les données de départ sont créées automatiquement au premier lancement.
Multijoueur	Plusieurs utilisateurs simultanés sur la même instance et la même base	Exigé pour le web et le mobile : « plusieurs utilisateurs jouent simultanément avec la même base de données et s'affrontent en temps réel ». Chaque utilisateur gère son équipe, tous passent par les mêmes services. Temps réel : **SignalR** (résultats de matchs et blessures diffusés aux clients connectés).
Import / export	Format **JSON** (CSV / XML possibles)	Service dédié appelé depuis le web et depuis l'API. Prévoir une base vide avant import.
Authentification	Hors périmètre de la V1	Demandé explicitement pour la suite.

Stratégie de migration vers MVC

- Migration **progressive** : l'application Blazor reste accessible (**/**) et le site MVC est servi sous **/mvc**. Rien n'est cassé pendant le développement.
- Le MVC **réutilise** les modèles, le contexte EF Core et les services existants : seules les vues et les contrôleurs sont nouveaux.
- Bascule finale prévue : MVC sur la racine du site, une fois toutes les pages migrées.

Organisation du code

Rôle MVC	Dossier	Contenu
Modèle	Models/ · Data/ · Services/	Team, Player, Position, Match, Injury · DbContext EF Core et données de départ · logique métier (simulation, blessures)
Vue	Views/	Layout Bootstrap commun (barre de navigation), vues Razor par contrôleur (Teams, Players)
Contrôleur	Controllers/	TeamsController, PlayersController : reçoivent la requête, appellent un service, renvoient une vue

2. Preuves de fonctionnement de l'application actuelle (bloc 2)

Application Blazor Server telle qu'elle fonctionnait pour le bloc 2, démarrée avec le jeu de données initial (3 équipes, 18 joueurs, 5 postes, 2 matchs, 1 blessure). Les captures ci-dessous montrent chaque écran, puis une simulation de match avec génération aléatoire de blessure. Ces captures ont été réalisées avant le passage à MySQL ; le même comportement (affichage, simulation de match, blessure aléatoire) a ensuite été revérifié sur la base MySQL.

Écran	Fonctionnalités démontrées
Dashboard	Compteurs (équipes, joueurs, blessés, matchs joués), top 5 des joueurs, derniers matchs
Teams	Création, modification et suppression d'équipes (nom, ville, entraîneur)
Players	Création, modification et suppression de joueurs : équipe, poste, vitesse, endurance, force, technique, note globale calculée, état blessé / apte
Positions	Gestion des postes et des caractéristiques attendues pour chacun
Matches	Création d'un match et **simulation** : score généré selon la force des joueurs valides
Injuries	**Blessures aléatoires** créées après un match, durée de récupération, bouton de rétablissement

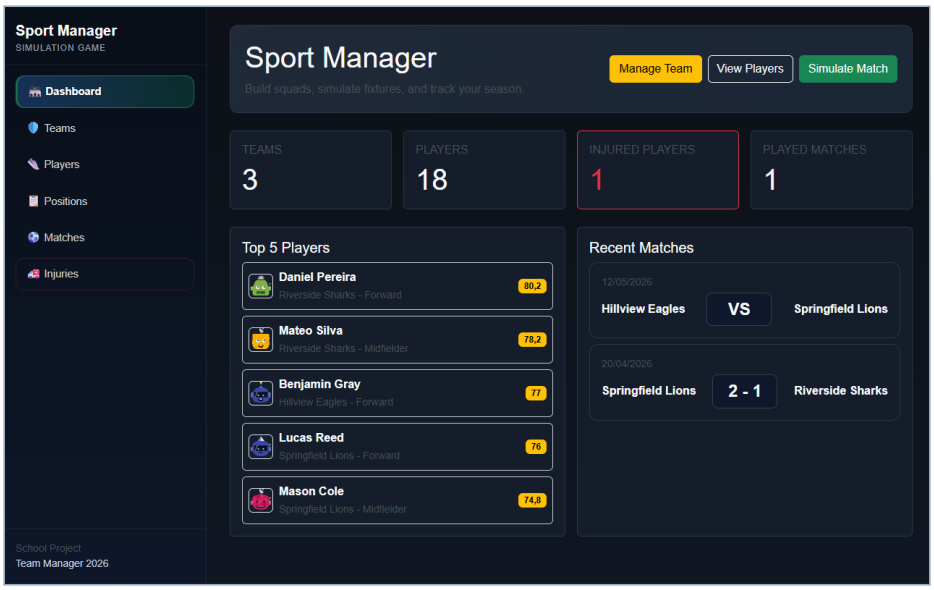


Figure 1 — Dashboard : état initial (3 équipes, 18 joueurs, 1 joueur blessé, 1 match joué)

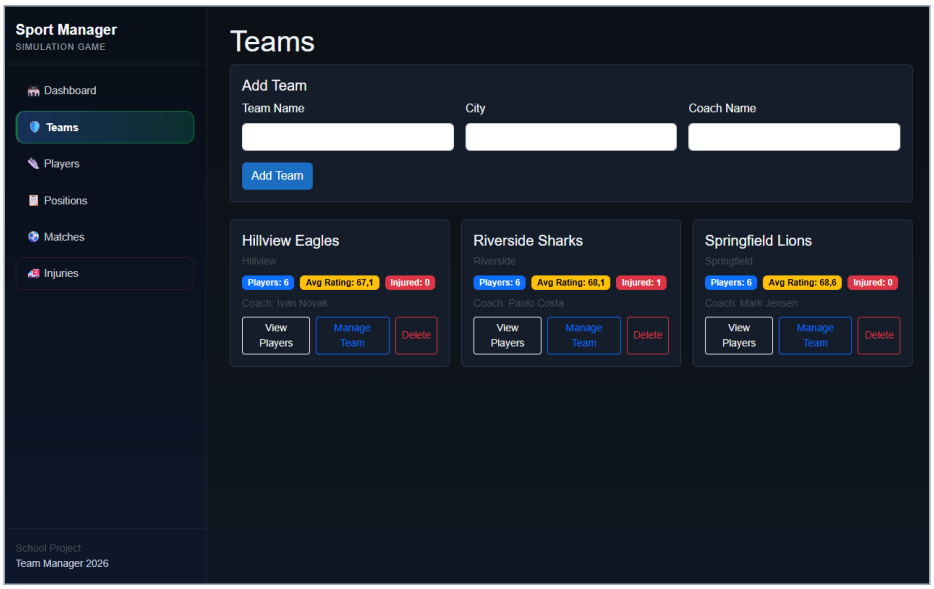


Figure 2 — Gestion des équipes

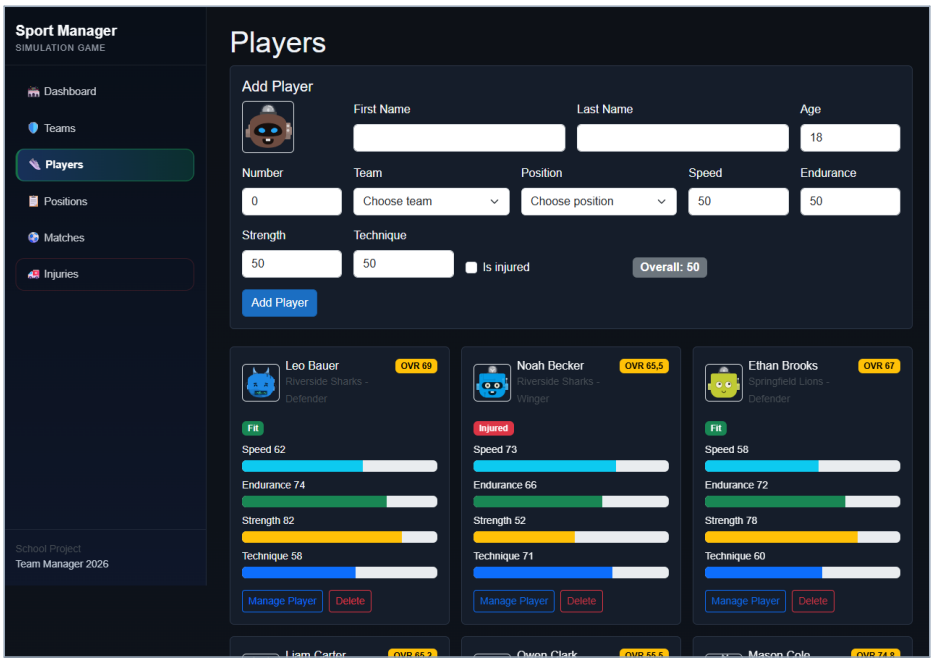


Figure 3 — Gestion des joueurs : caractéristiques, note globale (OVR) et état de chaque joueur (extrait)

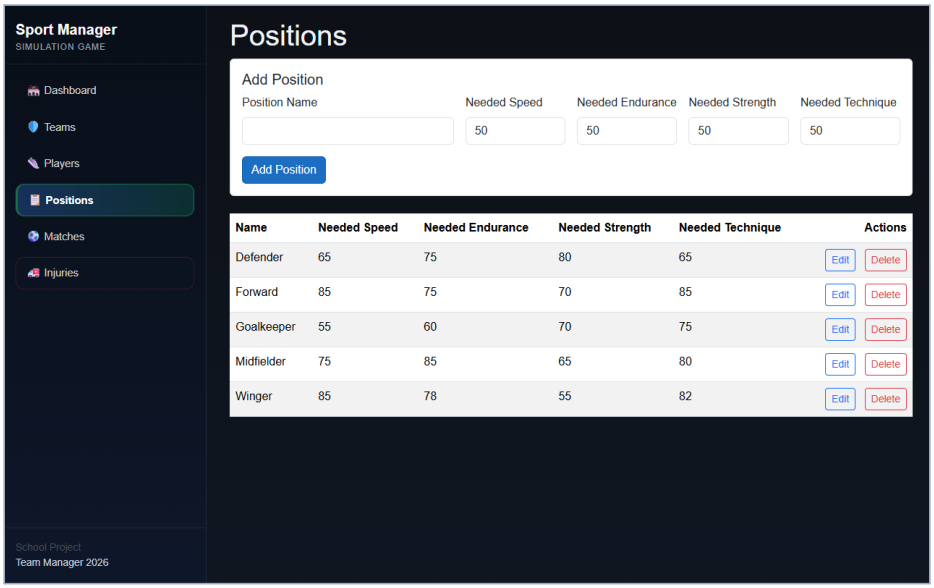


Figure 4 — Gestion des postes

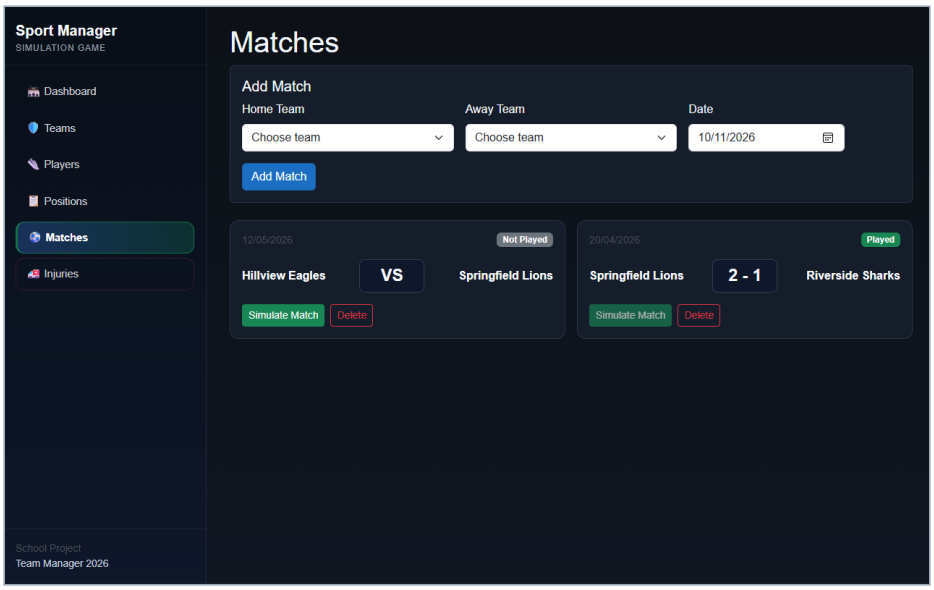


Figure 5 — Matchs avant simulation : le match Hillview Eagles / Springfield Lions n'est pas joué

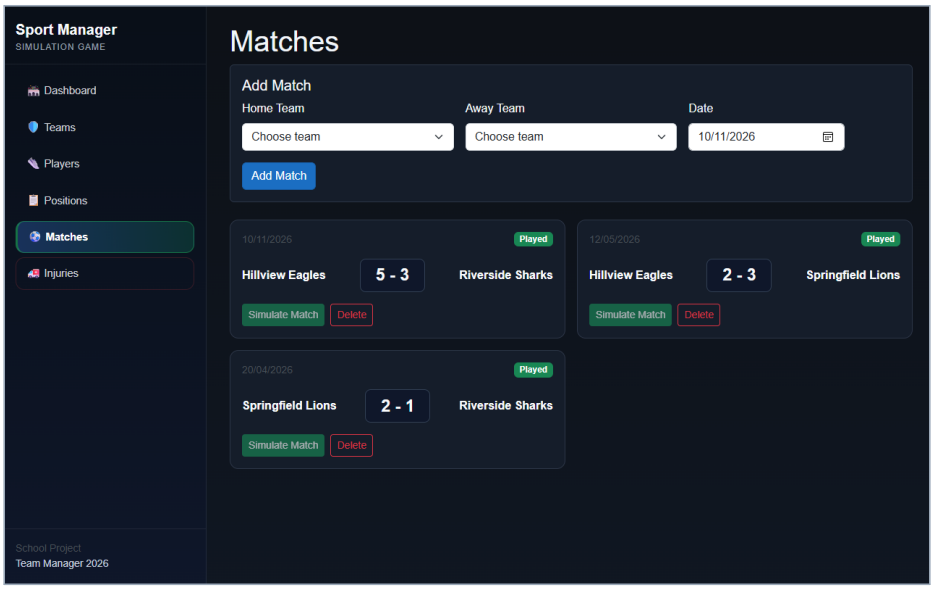


Figure 6 — Matchs après simulation : scores générés, matchs marqués « Played »

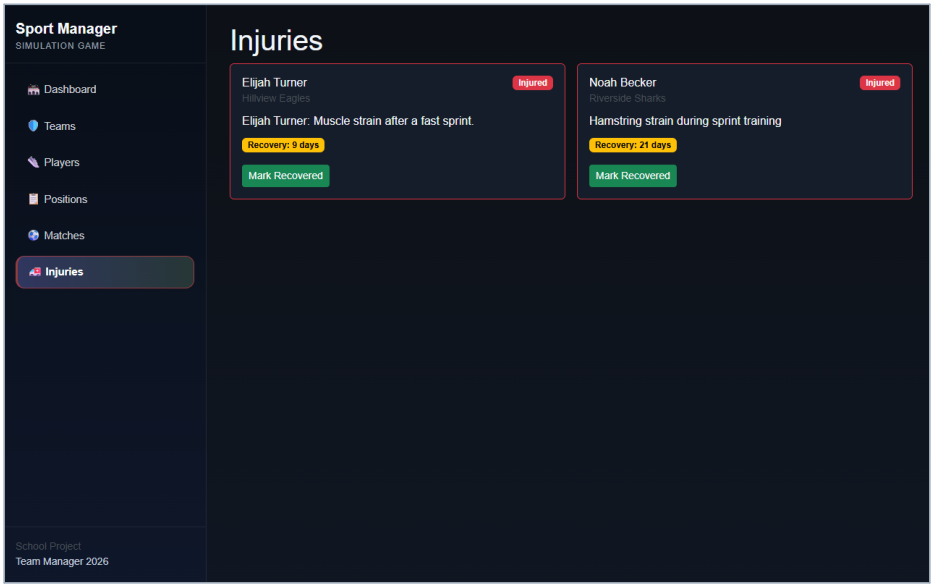


Figure 7 — Blessures : une nouvelle blessure aléatoire a été créée par la simulation (Elijah Turner, 9 jours), en plus de la blessure initiale

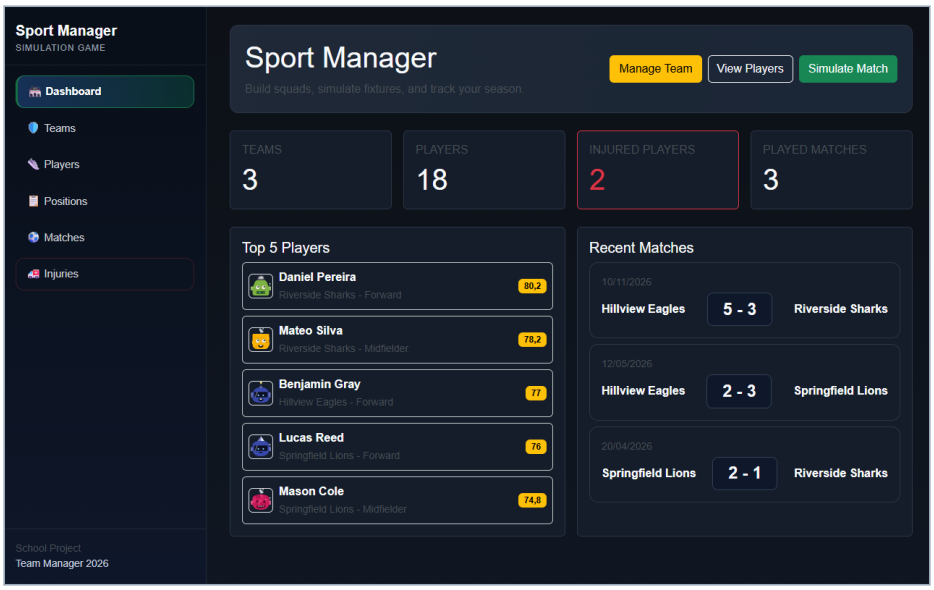


Figure 8 — Dashboard après simulation : les compteurs sont mis à jour

3. Version 1 de l'application web MVC

La V1 liste les équipes et les joueurs et affiche le détail d'une équipe et d'un joueur, sans authentification. Elle est accessible sous **/mvc** et se parcourt de page en page grâce à des liens.

Fonctionnalité	URL	Contrôleur / action	Vue
Liste des équipes	/mvc	TeamsController.Index	Views/Teams/Index.cshtml
Détail d'une équipe (infos + joueurs)	/mvc/Teams/Details/{id}	TeamsController.Details	Views/Teams/Details.cshtml
Liste des joueurs	/mvc/Players	PlayersController.Index	Views/Players/Index.cshtml
Détail d'un joueur (caractéristiques + équipe)	/mvc/Players/Details/{id}	PlayersController.Details	Views/Players/Details.cshtml

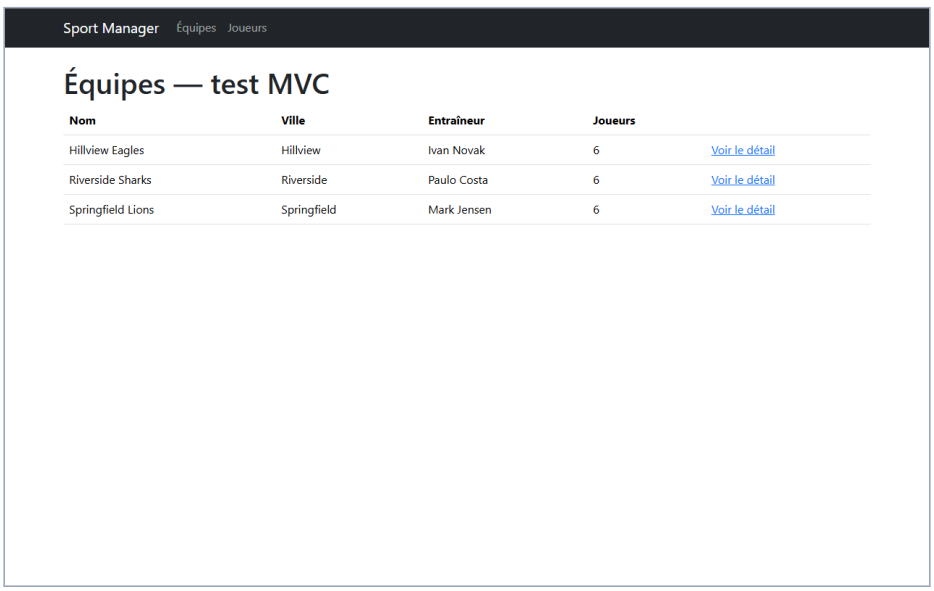


Figure 9 — Liste des équipes : nom, ville, entraîneur, nombre de joueurs, lien vers le détail

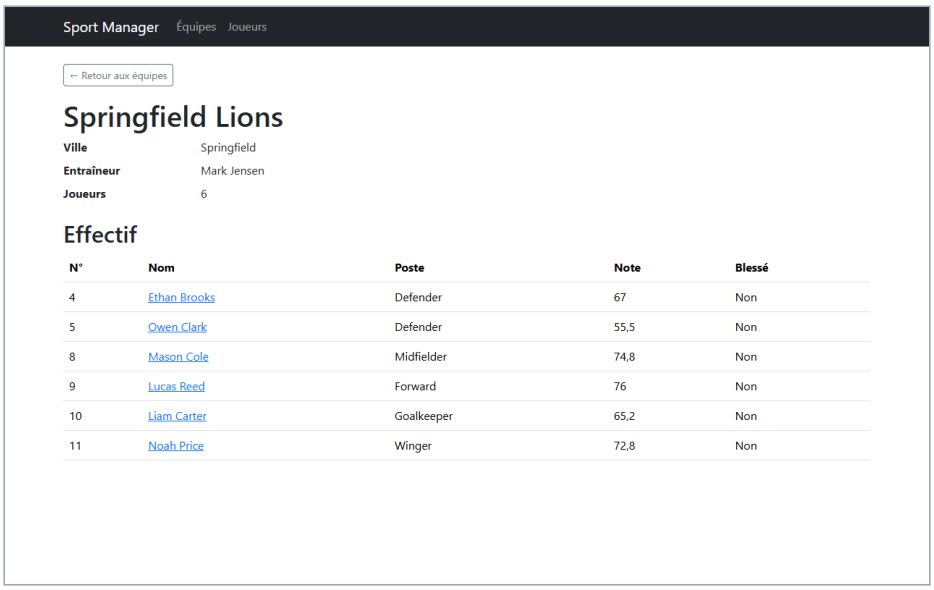


Figure 10 — Détail d'une équipe : informations de l'équipe et effectif, chaque joueur est cliquable

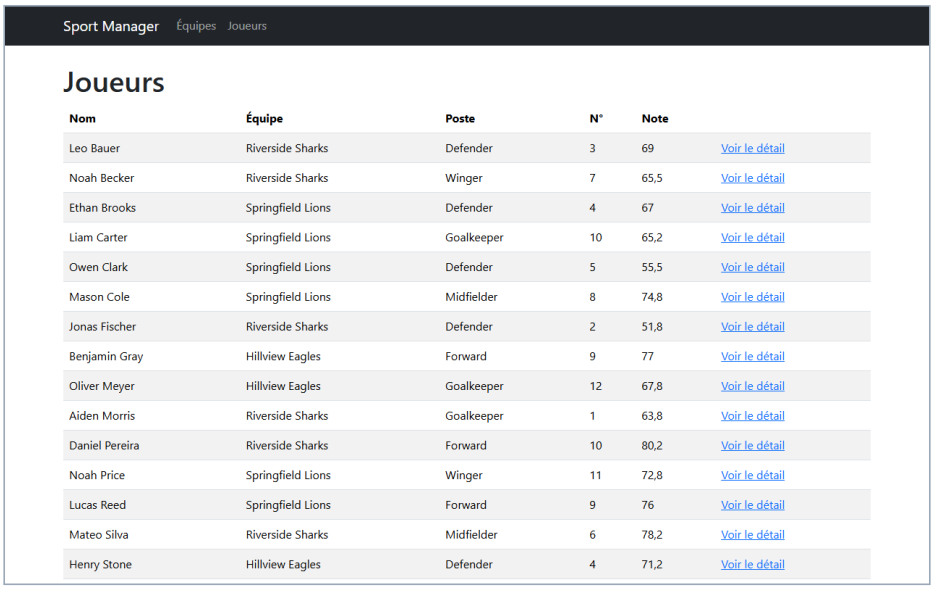


Figure 11 — Liste des joueurs : équipe, poste, numéro, note globale

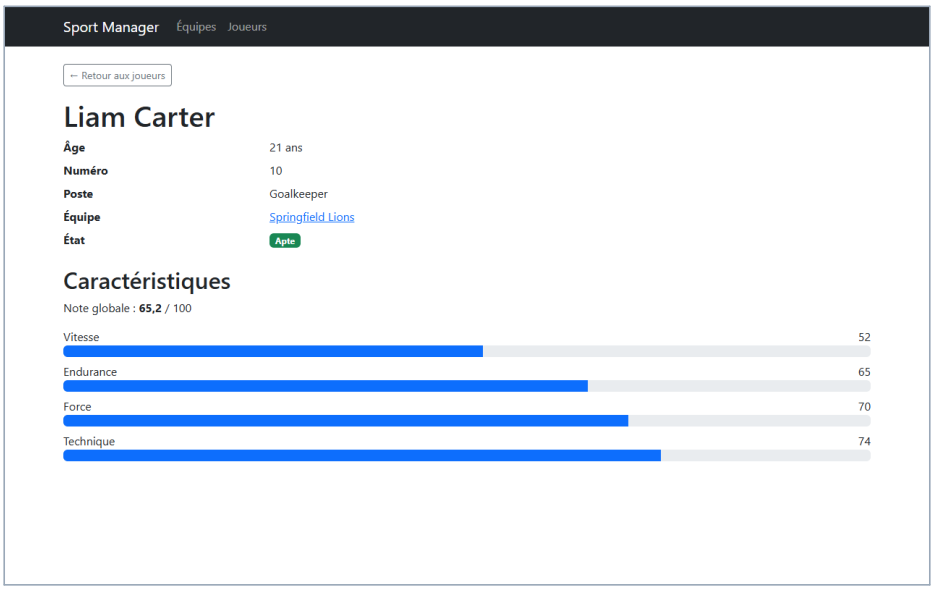


Figure 12 — Détail d'un joueur : informations, état, caractéristiques (vitesse, endurance, force, technique) et lien vers son équipe

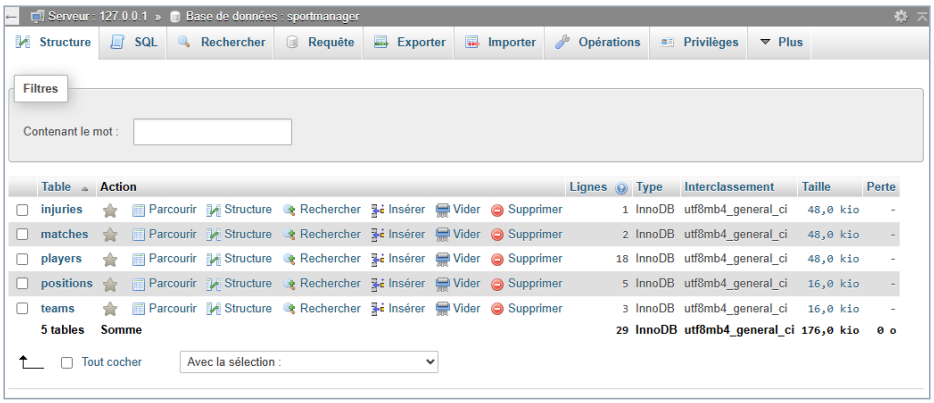


Figure 13 — Base « sportmanager » dans phpMyAdmin : 5 tables créées par l'application et remplies par les données de départ (3 équipes, 18 joueurs, 5 postes, 2 matchs, 1 blessure)

Vérifications effectuées

- Le projet compile sans erreur ni avertissement.
- Les 3 pages de détail d'équipe et les 18 pages de détail de joueur répondent correctement.
- Un identifiant inexistant (par exemple /mvc/Teams/Details/999) renvoie une erreur 404 au lieu de planter.
- L'application Blazor existante continue de fonctionner à côté du MVC.
- **Base de données :** migration de SQLite vers MySQL (phpMyAdmin) : au démarrage, l'application crée les 5 tables et insère les données de départ sans doublon, même après un redémarrage.
- Les pages MVC et Blazor (dashboard, matchs) répondent correctement sur MySQL, et la simulation de match avec blessure aléatoire s'enregistre bien dans la base.
- L'alerte de sécurité NuGet NU1903 (SQLite) a disparu avec la suppression de SQLite.

4. Prochaines étapes et points à valider

Planning prévisionnel

Étape	Contenu
1	Compléter le MVC : création, modification, suppression des équipes, postes et joueurs (CRUD), puis matchs, simulation et blessures
2	Page d'erreur MVC et amélioration de l'interface
3	API REST : mêmes fonctionnalités exposées en JSON
4	Application mobile .NET MAUI branchée sur l'API
5	Multijoueur sur la même instance : comptes utilisateurs, une équipe par utilisateur, matchs entre utilisateurs, diffusion en temps réel (SignalR)
6	Import / export des données (JSON, éventuellement CSV et XML)
7	Jeu de données cohérent, tests de non-régression, préparation de la soutenance

Questions à valider avec le responsable

- **Mobile :** .NET MAUI est-il accepté comme technologie, ou une autre est-elle attendue ?
- **Multijoueur :** l'énoncé parle de s'affronter « en temps réel » : un affichage mis à jour automatiquement (SignalR) suffit-il, ou faut-il un match synchrone entre deux utilisateurs ?
- **Blazor :** l'application Blazor doit-elle être conservée après la bascule vers MVC, ou remplacée ?
- **Authentification :** quand doit-elle être introduite (comptes joueurs, rôles) ?